

Q12: Corner Detection in Textured Regions

Course: Computer Vision (ITA05) | Assessment Tool 2: Case Study | CO3

Scenario

A system applies corner detection in highly textured regions, resulting in excessive keypoints. Analyze the issue and suggest methods to improve feature selection.

Analysis

Corner detectors such as Harris and Shi-Tomasi (`cv2.goodFeaturesToTrack`) score every pixel by how much the local intensity changes in all directions (using the eigenvalues of the structure tensor / second-moment matrix). In a highly textured region -- e.g. a checkerboard pattern, dense foliage, or fine repeating patterns -- almost every pixel has strong intensity variation in multiple directions, so a large fraction of the region scores highly as a 'corner'.

If the detector's quality threshold is set too low and no minimum spacing between corners is enforced, the detector reports thousands of redundant, closely-packed keypoints in the textured region while contributing little extra useful information -- neighbouring keypoints describe almost identical local neighbourhoods. This wastes computation in later stages (descriptor computation, matching) and can dominate a fixed feature budget, starving other, more structurally meaningful parts of the image (like the smooth region's real corners) of representation.

In the experiment, a naive detector with `qualityLevel=0.001` and `minDistance=1` reported 4186 corners, almost entirely clustered inside the textured half of the image, essentially treating texture noise as meaningful structure.

Suggested Solution / Improvements

- Raise the quality threshold (`qualityLevel` parameter). This keeps only corners whose 'corneriness' score is a significant fraction of the strongest corner in the image, filtering out the many weak, redundant corners produced by fine texture.
- Enforce a minimum distance between detected corners (`minDistance` parameter). This performs non-maximum suppression, so that only the strongest corner within a local neighbourhood survives, giving a well spread-out set of keypoints instead of a dense cluster.
- Cap the total number of corners (`maxCorners`) so that only a fixed budget of the strongest, best-distributed corners is retained -- this also bounds computational cost for later descriptor/matching stages.
- Optionally, apply adaptive/grid-based feature selection (divide the image into cells and select the top-N corners per cell) to guarantee spatial coverage rather than corners clustering only in texture-rich areas.

Output / Result Screenshot

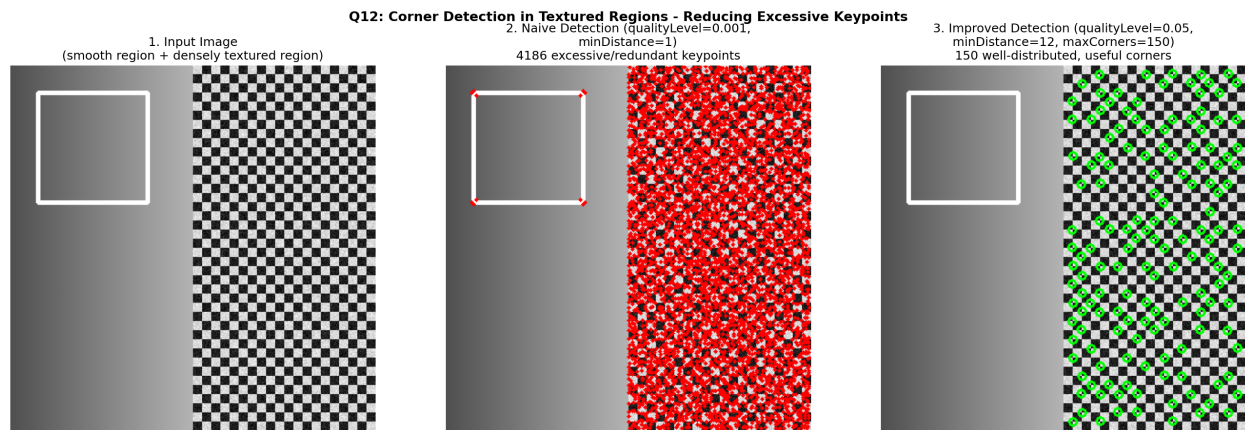


Figure: Output generated by Q12_corner_detection_textured_regions.py

Conclusion

With qualityLevel raised to 0.05, minDistance set to 12 pixels, and maxCorners capped at 150, the detector produced 150 well-distributed, genuinely useful corners spread across both the smooth and textured regions -- compared to 4186 redundant corners from the naive settings -- demonstrating that proper thresholding and non-maximum suppression are essential for reliable feature selection in textured regions.

Implementation (Python / OpenCV)

Full source code is provided separately as **Q12_corner_detection_textured_regions.py**. The key implementation is reproduced below.

```
"""
Question 12: Corner Detection in Textured Regions
-----
A system applies corner detection in highly textured regions, resulting
in excessive keypoints. This script demonstrates the problem with a raw
Harris/Shi-Tomasi detector and the fix using quality thresholding,
minimum-distance non-maximum suppression, and restricting detection to
a fixed keypoint budget.

Pipeline:
1. Build a synthetic image with one smooth region and one densely
   textured (checker/noise) region.
2. Run Shi-Tomasi corner detection with a LOW quality threshold and NO
   minimum distance -> excessive, redundant keypoints in the textured
   region (the problem).
3. Run Shi-Tomasi again with a proper quality threshold, a minimum
   distance between corners (non-maximum suppression), and a capped
   corner budget -> a clean, well-distributed, useful set of corners
   (the solution).
4. Save a comparison figure as the output screenshot.
"""

import cv2
import numpy as np
import matplotlib.pyplot as plt

def build_textured_image(size=400):
    """Half smooth gradient, half densely textured checker + noise pattern."""
    img = np.zeros((size, size), dtype=np.uint8)

    # Left half: smooth gradient (few real corners)
    for x in range(size // 2):
        img[:, x] = int(80 + 100 * (x / (size // 2)))

    # Right half: dense checkerboard + noise -> highly textured region
    tile = 10
    for y in range(size):
        for x in range(size // 2, size):
            if ((x // tile) + (y // tile)) % 2 == 0:
                img[y, x] = 220
            else:
                img[y, x] = 30

    rng = np.random.default_rng(1)
    noise = rng.normal(0, 15, (size, size // 2)).astype(np.int16)
    right = img[:, size // 2:].astype(np.int16) + noise
    img[:, size // 2:] = np.clip(right, 0, 255).astype(np.uint8)

    cv2.rectangle(img, (30, 30), (150, 150), 255, 3) # a real structural corner shape
    return img

def detect_corners_naive(gray_img):
    """Excessive corner detection: low quality level, no min distance."""
    corners = cv2.goodFeaturesToTrack(
        gray_img, maxCorners=5000, qualityLevel=0.001, minDistance=1
    )
    vis = cv2.cvtColor(gray_img, cv2.COLOR_GRAY2BGR)
    n = 0 if corners is None else len(corners)
    if corners is not None:
        for c in corners:
            x, y = c.ravel()
            cv2.circle(vis, (int(x), int(y)), 2, (0, 0, 255), -1)
    return vis, n

def detect_corners_improved(gray_img):
    """Controlled corner detection: proper quality threshold, min distance, budget."""
    corners = cv2.goodFeaturesToTrack(
        gray_img, maxCorners=150, qualityLevel=0.05, minDistance=12
    )
    vis = cv2.cvtColor(gray_img, cv2.COLOR_GRAY2BGR)
    n = 0 if corners is None else len(corners)
    if corners is not None:
        for c in corners:
            x, y = c.ravel()
            cv2.circle(vis, (int(x), int(y)), 4, (0, 255, 0), 2)
    return vis, n

def main():
```

```

img = build_textured_image()

naive_vis, n_naive = detect_corners_naive(img)
improved_vis, n_improved = detect_corners_improved(img)

print(f"Corners detected (naive, low threshold, no min-distance): {n_naive}")
print(f"Corners detected (improved, thresholded + NMS + budget): {n_improved}")

fig, axes = plt.subplots(1, 3, figsize=(18, 6))

axes[0].imshow(img, cmap='gray')
axes[0].set_title('1. Input Image\n(smooth region + densely textured region)')
axes[0].axis('off')

axes[1].imshow(cv2.cvtColor(naive_vis, cv2.COLOR_BGR2RGB))
axes[1].set_title(f'2. Naive Detection (qualityLevel=0.001,\nminDistance=1)\n{n_naive} excessive/redundant keypoints')
axes[1].axis('off')

axes[2].imshow(cv2.cvtColor(improved_vis, cv2.COLOR_BGR2RGB))
axes[2].set_title(f'3. Improved Detection (qualityLevel=0.05,\nminDistance=12, maxCorners=150)\n{n_improved} well-distributed')
axes[2].axis('off')

plt.suptitle('Q12: Corner Detection in Textured Regions - Reducing Excessive Keypoints',
             fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig('/home/claude/cv_solutions/images/Q12_output.png', dpi=150, bbox_inches='tight')
print("Saved output screenshot to Q12_output.png")

if __name__ == "__main__":
    main()

```